

CSS Grid

The 2-Dimensional Layout System

explained from first principles

A complete, corrected teaching handout: the mental model, rows & columns, the fr unit, line-based placement, named areas, code you can paste, diagrams, and common mistakes.

In this session

1. What Grid is — and the two-dimensional idea
2. Rows, columns & the questions to ask first
3. `grid-template-columns / rows + repeat()`
4. The `fr` unit — how free space is shared (with correct math)
5. `gap`, and sizing tracks the right way
6. Placing items: grid lines, `grid-column / grid-row`, `span`
7. `grid-template-areas` — layouts you can read
8. Grid vs Flexbox — when to use which (and both)
9. Common mistakes + a practice game
10. Mini-project: a Dashboard layout with line-based placement

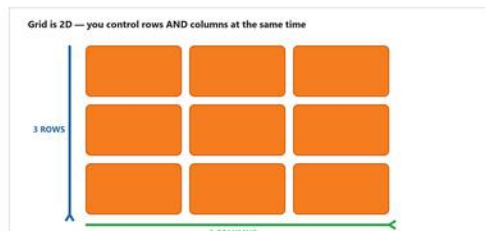
THE BIG PICTURE

What is CSS Grid, really?

Flexbox lays things out in ONE direction at a time — a row or a column. CSS Grid is the tool for TWO dimensions at once: you define rows AND columns together and place items into that structure. It is the most powerful layout system CSS has ever had, and it is built specifically for page-level and component-level layouts.

First principle

Grid is a TWO-DIMENSIONAL layout system. You draw a grid of rows and columns on the container, then drop items into cells (or make them span several cells). Flexbox = one line at a time; Grid = a whole table-like structure you control precisely.



You set both the rows and the columns — that's the 2D idea.

The mental model: the container is graph paper. You decide how many columns and rows it has and how big each track is. Every direct child then occupies one or more cells. You mostly configure the container; individual items can opt to span or move to specific lines.

Why Grid exists (your notes, sharpened)

- Built for complex, 2D layouts: multiple rows and columns, irregular spacing, overlap.
- Removes wrapper-div soup. Many layouts that needed nested wrappers with floats become one grid container with direct children.
- Design approach: start small (a 2- or 3-column grid) then grow — but also think 'outside-in': define the page skeleton first, fill details later.

The one rule to internalize

Grid = STRUCTURE (the skeleton of the page). Flexbox = ALIGNMENT (arranging items inside a region). Use Grid for the page layout, and Flexbox inside each grid area to align that area's contents. They are partners, not rivals.

The one line that starts everything

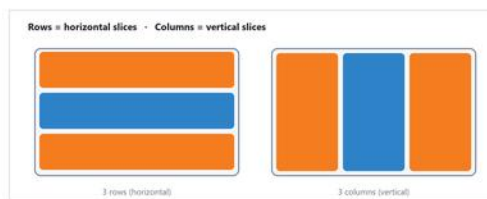
```
/* turn any element into a grid container */
.container {
  display: grid; /* its direct children become grid items */
}
```

THE CORE QUESTIONS

Rows, columns & what to ask first

Every time you build a grid, answer three questions before writing anything else. Your notes had this exactly right — it is the fastest way to keep a grid under control:

- How many COLUMNS do I want, and how wide is each?
- How many ROWS do I want, and how tall is each?
- How much GAP between them?



Rows are horizontal slices; columns are vertical slices.

You answer the first two with `grid-template-columns` and `grid-template-rows`, and the third with `gap`.

`grid-template-columns` / `grid-template-rows`

These list the SIZE of each track (a track = one column or one row). List three values and you get three columns; the widths can be pixels, percentages, fr units, auto, or min/max functions.

```
.container {
  display: grid;
  grid-template-columns: 90px 90px 90px 90px 90px 90px;
  /* six 90px columns */
}
```

`repeat()` — don't type the same value six times

When tracks repeat, `repeat()` is shorter and clearer. These two are identical:

```
grid-template-columns: 90px 90px 90px 90px 90px 90px;
grid-template-columns: repeat(6, 90px); /* same thing */
```

Correction to a common note

`grid-template-ROWS` defines the number and HEIGHT of rows, and `grid-template-COLUMNS` defines the WIDTH of columns — not the other way round. Also: 'grid-template-row/column' (singular) is not the real property name; it is `grid-template-rows` / `grid-template-columns` (plural).

THE UNIT THAT MAKES GRID RESPONSIVE

The fr unit

fr means 'fraction of the free space'. Instead of hard-coding pixel widths, you tell the browser how to SHARE the available space. This is what makes a grid adapt to the screen without media queries.



1fr means one share. With three 1fr columns, the row is split into three equal shares. As the container gets wider or narrower, each fr recomputes automatically — the ratios stay, the pixels change.

Getting the fr math exactly right

fr divides the FREE space — the space left AFTER fixed tracks and all gaps are subtracted. When every track is an fr and there are no gaps, it simplifies to: each track = (container width / sum of fr values) × its own fr.



Your example, corrected & clarified

Container 600px, columns 1fr 2fr 3fr → total 6 parts → one part = 600/6 = 100px. So 1fr=100px, 2fr=200px, 3fr=300px. Correct! The subtle bit your notes should add: if there were also a fixed 120px column or a 20px gap, fr would divide (600 - 120 - gaps), NOT the full 600. fr always shares what's LEFT.

```
/* 600px container, no gaps */
grid-template-columns: 1fr 2fr 3fr; /* → 100px 200px 300px */

/* mix fixed + fr: sidebar fixed, main takes the rest */
grid-template-columns: 240px 1fr; /* main = 600 - 240 = 360px */
```

Sizing tracks the right way

- Prefer fr and content-based sizing over fixed pixel columns — it stays responsive.
- Avoid fixed heights on rows where you can. Let content decide height (auto), or use minmax() for a floor/ceiling.
- minmax(200px, 1fr) = 'at least 200px, but grow to a fraction of free space'. Extremely common for responsive columns.

```
/* responsive card grid — as many 220px+ columns as fit */
grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
```

SPACING & PLACEMENT

gap, and placing items on lines

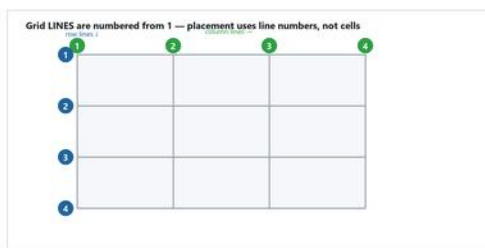
gap — the space between tracks

gap sets the gutter between rows and columns, without adding space on the outer edges. One value sets both; two values set row-gap and column-gap.

```
.container {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  gap: 16px;           /* row & column gap */  
  /* gap: 12px 24px; row-gap column-gap */  
}
```

Grid LINES — the key mental shift

Placement in Grid is based on LINES, not cells. A grid with 3 columns has 4 vertical lines (numbered 1 to 4), and a grid with 3 rows has 4 horizontal lines. You place an item by saying which line it STARTS on and which line it ENDS on.



Why 'lines, not cells' matters

An item from column line 1 to column line 4 spans all three columns. Beginners count cells and get off-by-one errors; count LINES instead. n columns always have $n+1$ column lines.

grid-column / grid-row — place and span

The long form is grid-column-start / grid-column-end (and the row equivalents). The shorthand grid-column and grid-row take start / end in one line. This is what your notes were reaching for:

```
/* long form */  
.item {  
  grid-row-start: 1;  
  grid-row-end: 3;    /* rows 1-3 (spans 2 rows) */  
  grid-column-start: 1;  
  grid-column-end: 4; /* columns 1-4 (spans 3 cols) */  
}  
  
/* shorthand — identical result */  
.item {  
  grid-row: 1 / 3;  
  grid-column: 1 / 4;  
}  
  
/* or say how many tracks to span */  
.item { grid-column: 1 / span 3; }
```

Placing an item across lines — grid-column: 1 / 4 spans columns 1–4



PLACEMENT, CONTINUED

grid-area & named areas

The grid-area shorthand — mind the order

Important correction to your notes

You wrote 'grid-area: row-start col-start row-end col-end'. The correct order is: grid-area: row-start / column-start / row-end / column-end — separated by SLASHES, and the pairs are (row, column), (row, column). It is start-corner then end-corner. Easy to get backwards, so double-check.

```
/* row-start / col-start / row-end / col-end */  
.item { grid-area: 1 / 1 / 3 / 4; }  
/* same as: grid-row: 1 / 3; grid-column: 1 / 4; */
```

grid-template-areas — the most readable layout tool

Instead of counting lines, you can NAME regions and draw the layout as ASCII art. This is the feature that makes Grid feel magical for page layouts — the CSS literally looks like the page.



Name the regions, then assign each element to one

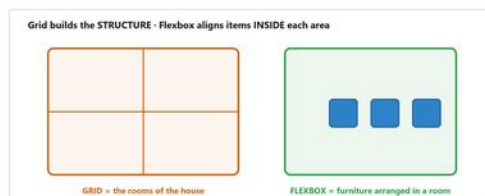
```
.page {  
  display: grid;  
  grid-template-columns: 200px 1fr;  
  grid-template-rows: auto 1fr auto;  
  grid-template-areas:  
    "header header"  
    "sidebar main"  
    "footer footer";  
  min-height: 100vh;  
}  
.header { grid-area: header; }  
.sidebar { grid-area: sidebar; }  
.main { grid-area: main; }  
.footer { grid-area: footer; }
```

Why students love this

To move the sidebar to the right, you just rewrite the ASCII rows — no line numbers to recount. Repeat a name across cells to make an area span. Use a dot (.) for an empty cell.

CHOOSING THE RIGHT TOOL

Grid vs Flexbox — and using both



They solve different problems. The house analogy from your notes is perfect:

Grid = the rooms the overall 2D structure of the page — where header, sidebar, main, footer sit.

Flexbox = the furniture arranging and aligning the items INSIDE one room (a nav bar's links, a card's buttons).

Rules of thumb

- Laying out a whole page or a 2D component (rows AND columns)? → Grid.
- Aligning a single row or column of items, or distributing space along one axis? → Flexbox.
- Most real UIs use BOTH: a Grid for the page skeleton, Flexbox inside each area.

```
.page { display: grid; grid-template-columns: 200px 1fr; }  
.navbar { display: flex; justify-content: space-between; align-items: center; }  
/* Grid places the navbar area; Flexbox arranges what's inside it */
```

WATCH OUT FOR THESE

Common mistakes & recap

- Counting cells instead of LINES when placing items. n tracks = $n+1$ lines; `grid-column: 1 / 4` spans 3 columns.
- Getting grid-area order wrong. It's row-start / col-start / row-end / col-end, slash-separated.
- Thinking `fr` divides the whole width. It divides the FREE space left after fixed tracks and gaps.
- Hard-coding pixel heights on rows. Prefer `auto` or `minmax()` so content isn't clipped.
- Reaching for Grid to align a single row of items — that's Flexbox's job.
- Property name slips: it's `grid-template-columns` / `-rows` (plural), `display: grid` on the CONTAINER.

30-second recap

display: grid makes a container; direct children become grid items.

grid-template-columns / **-rows** define track count and sizes.

repeat(n, size) shorthand for repeating tracks.

fr share of the free space (responsive).

gap gutters between tracks.

grid-column / **grid-row** place/span an item by line numbers (start / end).

grid-template-areas name regions and draw the layout as text.

Practice

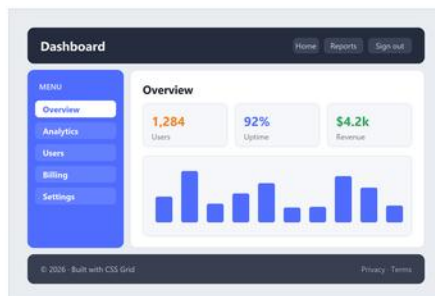
Play the CSS Grid game: cssgridgarden.com — it teaches columns, rows, line numbers and spanning by watering a carrot garden. Pair it with Flexbox Froggy and students will have both systems in muscle memory.

MINI-PROJECT — BUILD IT YOURSELF

A Dashboard layout with Grid

Let's build a real page skeleton with Grid: a header across the top, a sidebar on the left, the main content area, and a footer — the classic 'Holy Grail' layout. We'll place each region by GRID LINE numbers (grid-row / grid-column: start / end) — the line-based method.

The finished result



This is exactly what the code below produces.

MINI-PROJECT

Step 1 — the HTML skeleton

Plain HTML with clear class names. Four regions, no wrapper-div soup. Save as index.html.

index.html — structure only

```
<div class="page">
  <header class="header">Dashboard</header>
  <aside class="sidebar">Menu</aside>
  <main class="main">Overview + charts</main>
  <footer class="footer">© 2026</footer>
</div>
```

Step 2 — define the grid tracks

The container becomes a grid: a fixed 200px sidebar column and a flexible main column (so two columns → column lines 1, 2, 3). Three rows — header and footer hug their content, the middle row takes the remaining height (so three rows → row lines 1, 2, 3, 4). No areas here — we place items by line number next.

```
.page {
  display: grid;
  grid-template-columns: 200px 1fr; /* col lines: 1 | 2 | 3 */
  grid-template-rows: auto 1fr auto; /* row lines: 1 2 3 4 */
  gap: 10px;
  min-height: 100vh;
}
```

Count the LINES, not the cells

2 columns give 3 vertical lines (1,2,3); 3 rows give 4 horizontal lines (1,2,3,4). Every item is placed by saying which line it STARTS on and which line it ENDS on. This map shows exactly where each region sits:



MINI-PROJECT

Step 3 — place each element by line number

Now the method you learned in class: grid-row and grid-column with start / end line numbers. The header and footer span BOTH columns by going from column line 1 to line 3. The sidebar and main sit side by side in the middle row.

Line-based placement — start line / end line

```
/* header: full width, first row */
.header {
  grid-row: 1 / 2;
  grid-column: 1 / 3; /* spans both columns */
}

/* sidebar: middle row, left column */
.sidebar {
  grid-row: 2 / 3;
  grid-column: 1 / 2;
}

/* main: middle row, right column */
.main {
  grid-row: 2 / 3;
  grid-column: 2 / 3;
}

/* footer: full width, last row */
.footer {
  grid-row: 3 / 4;
  grid-column: 1 / 3; /* spans both columns */
}
```

The long form is identical

grid-row: 1 / 2 is shorthand for grid-row-start: 1; grid-row-end: 2. You can also write grid-column: 1 / span 2 to say 'start at line 1 and span 2 columns' — handy when you'd rather count tracks than end-lines. Either way, header ends at column line 3, spanning the full width.

Step 4 — Flexbox INSIDE the areas

Grid built the rooms; now Flexbox arranges the furniture. The header is a flex row (title left, nav right); the stat cards are a flex row; the sidebar menu is a flex column.

```
.header {
  display: flex;
  justify-content: space-between; /* title left, nav right */
  align-items: center;
  padding: 0 20px;
}

.sidebar { display: flex; flex-direction: column; gap: 8px; }
.cards { display: flex; gap: 12px; }
.cards > * { flex: 1; } /* equal-width stat cards */
```

You just built a real page

Grid gave you the whole 2D skeleton in a handful of lines and zero wrapper divs; Flexbox aligned the contents of each region. This Grid-outside / Flexbox-inside pattern is exactly how modern layouts are built. Rebuild it once from memory — that's where it sticks.

Challenge: make it responsive. Below 700px, switch to a single column (grid-template-columns: 1fr) and restack the items with line numbers — header 1/2, main 2/3, sidebar 3/4, footer 4/5 — inside a media query, so everything reads top to bottom on mobile.